

Luis Mario Gaytan

Assignment-3 (Concurrency)

The goal of this assignment is to allow you to explore the concurrency abstractions like threads, locks, and condition variables, which are vastly used in real life software applications. Here we will build a Master-Worker Thread pool that would allow you to understand how real-world systems like web servers and database servers work.

Tasks

You must modify this skeleton code master-worker.c in the following ways.

1. You must add code to spawn the required number of worker threads, and write the function to be run by these threads.
2. This function will remove/consume items from the shared buffer and print them to screen.
3. You must add logic to correctly synchronize the producer and consumer threads in such a way that every number is produced and consumed exactly once.
4. Producers must not try to produce when the buffer is full, and consumers should not consume from an empty buffer.
5. The main thread must call `pthread_join` on the master and worker threads and terminate itself once all threads have joined.
6. Your solution must only use pthreads condition variables for waiting and signaling: busy waiting is not allowed

Assumptions:

- a) While you need to ensure that all C workers are involved in consuming the integers, it is not necessary to ensure perfect load balancing between the workers.
- b) Once all M integers (from 0 to M - 1) have been produced and consumed, all threads must exit.

```

1  master-worker.c
13 /*
12  *
11  * This program takes four command-line arguments:
10  * M - Total number of integers to produce (0 to M-1)
9   * N - Maximum size of the shared buffer
8   * C - number of workers threads
7   * P - number of master threads
6   *
5   * P producer thread that generate integers 0 ... M-1
4   * C consumer (worker) threads that take numbers from a shared buffer size N
3   * and print them. Each number must be produce exactly once and consumed exactly
2   * once. Producers must block if buffer is full; consumer must block if buffer is
1   * empty. Use only condition variables for waiting/signaling - no busy waiting.
   */
   #include <errno.h>
   #include <pthread.h>

```

```

// Synchronization Primitives
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t full =
    PTHREAD_COND_INITIALIZER; // Signaled when buffer has space
pthread_cond_t empty =
    PTHREAD_COND_INITIALIZER; // Signaled when buffer has data

int produced_count = 0; // total number produced so far (0...M)
int consumed_count = 0; // total number consumed so far (0...M)

int item_to_produce, curr_buf_size;
int total_items, max_buf_size, num_workers, num_masters;

int *buffer;

```

```

6  /*
5  * Producer must lock the mutex, wait while buffer is full (count == N)
4  * Produce the next number
3  * Insert into buffer
2  * Signal consumers that buffer is not empty
1  * Unlock mutex
   */
   void *producer(void *arg) {
       // TODO implement producer function.
       return NULL;
   }

```

```

8 /*
7  * Consumer must Lock mutex, wait for buffer empty (count == 0) and there are
6  * still numbers to consume When woken check if there is data. If buffer is
5  * empty but all numbers consumed, exit
4  * Remove a number from buffer
3  * Print the number
2  * Signal producer that buffer is not full
1  * Unlock mutex
0  */
void *consumer(void *arg) {
    // TODO implement consumer function
    return NULL;
}

```

```

// write function to be run by worker threads
// ensure that the workers call the function print consumed when they consume an
// item
// TODO Update main function
// Create both producer and consumer threads (P producer, C consumer)
// Join all threads after they finish
// Free resources
int main(int argc, char *argv[]) {
    int *master_thread_id;
}

```

```

void *generate_requests_loop(void *data) {
    int thread_id = *((int *)data);

    while (1) {
        if (item_to_produce >= total_items) {
            break;
        }

        buffer[curr_buf_size++] = item_to_produce;
        print_produced(item_to_produce, thread_id);
        item_to_produce++;
    }
    return 0;
}

```

I will use this part of the code as reference, will not be part of the final code but used inside of producer and consumer functions.

```
int total_items, max_buf_size, num_workers, num_masters;
```

I also prefer shorter names in this case, I have comments that give me the context needed to understand what a single letter means in this context.

```
int consumed_count = 0, // total number consumed so far
int M, N, C, P; // M(number to produce)
// N(buffer size)
// C(number of consumer)
// P(number of producer)
```

Better and will be faster to type logic.

```
for (i = 0; i < num_masters; i++)
    pthread_create(&master_thread[i], NULL, generate_requests_loop,
        (void *)&master_thread_id[i]);
```

I'll handle it differently.

```
void print_produced(int num, int master) {
    printf("Produced %d by master %d\n", num, master);
}
void print_consumed(int num, int worker) {
    printf("Consumed %d by worker %d\n", num, worker);
}
```

And also this print functions can be handle differently now that I have a Producer and Consumer function.

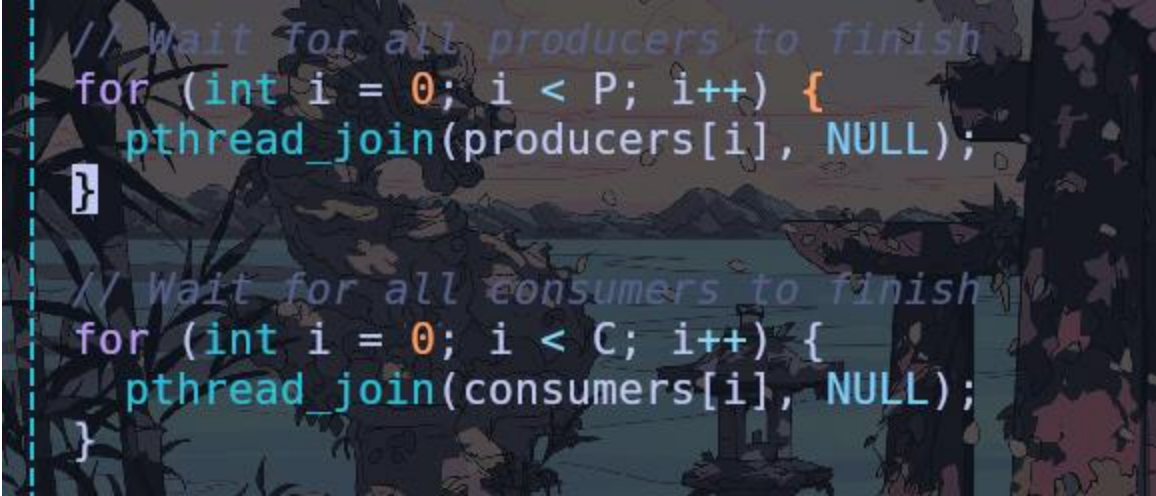
```
if (argc < 5) {
    printf("./master-worker #total_items #max_buf_size #num_workers #masters "
           "e.g. ./exe 10000 1000 4 3\n");
    exit(1);
} else {
    num_masters = atoi(argv[4]);
    num_workers = atoi(argv[3]);
    total_items = atoi(argv[1]);
    max_buf_size = atoi(argv[2]);
}
```

No need for the else, the arrangement of the values are crazy and the print statement just looks... interesting.

Ill just clean the code after implementing Producer and Consumer functions.

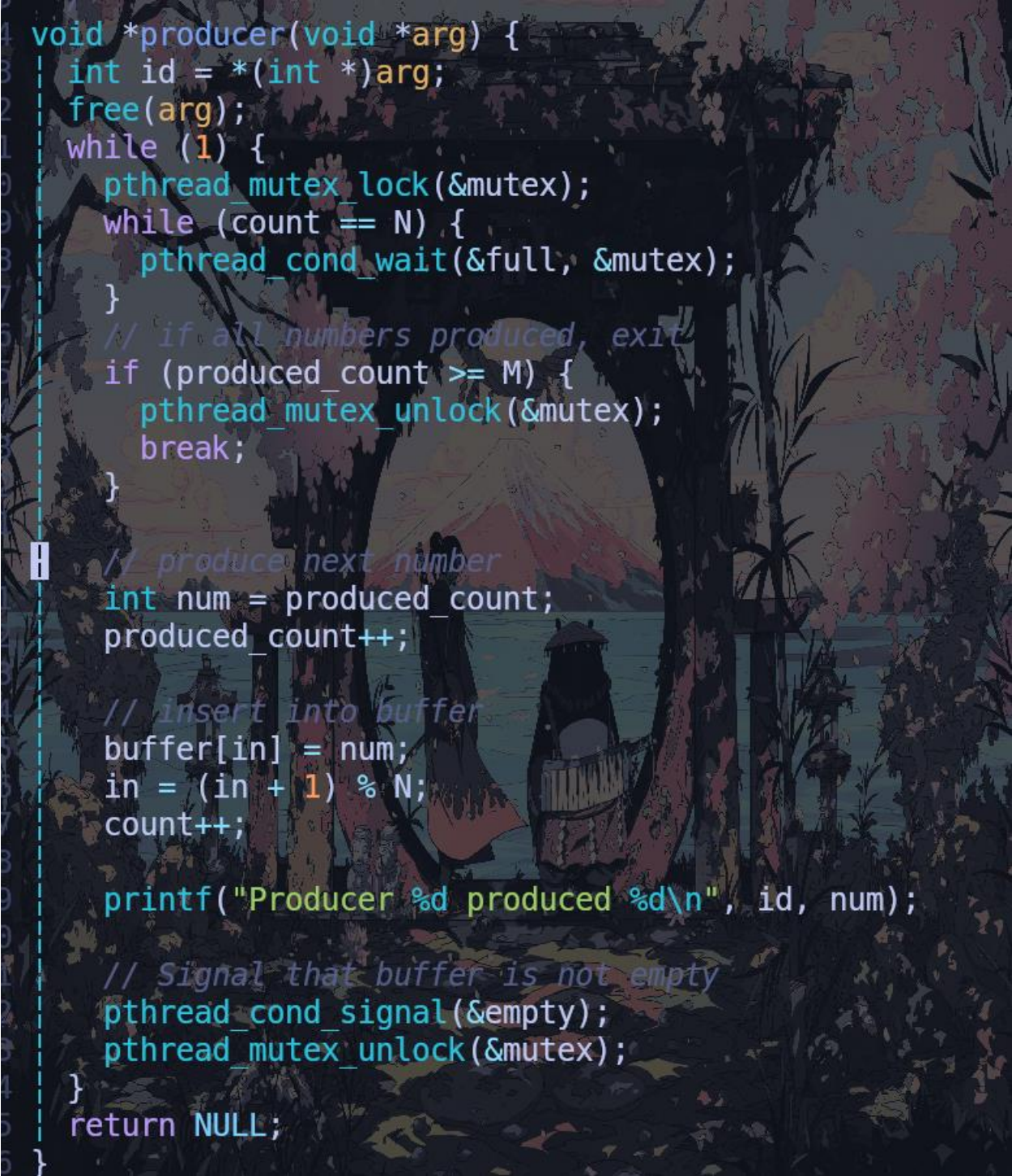
```
// Create producer threads
pthread_t producers[P];
for (int i = 0; i < P; i++) {
    int *id = (int *)malloc(sizeof(int));
    *id = i;
    if (pthread_create(&producers[i], NULL, producer, id) != 0) {
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }
}

// Create consumer threads
pthread_t consumers[C];
for (int i = 0; i < C; i++) {
    int *id = (int *)malloc(sizeof(int));
    *id = i;
    if (pthread_create(&consumers[i], NULL, consumer, id) != 0) {
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }
}
```



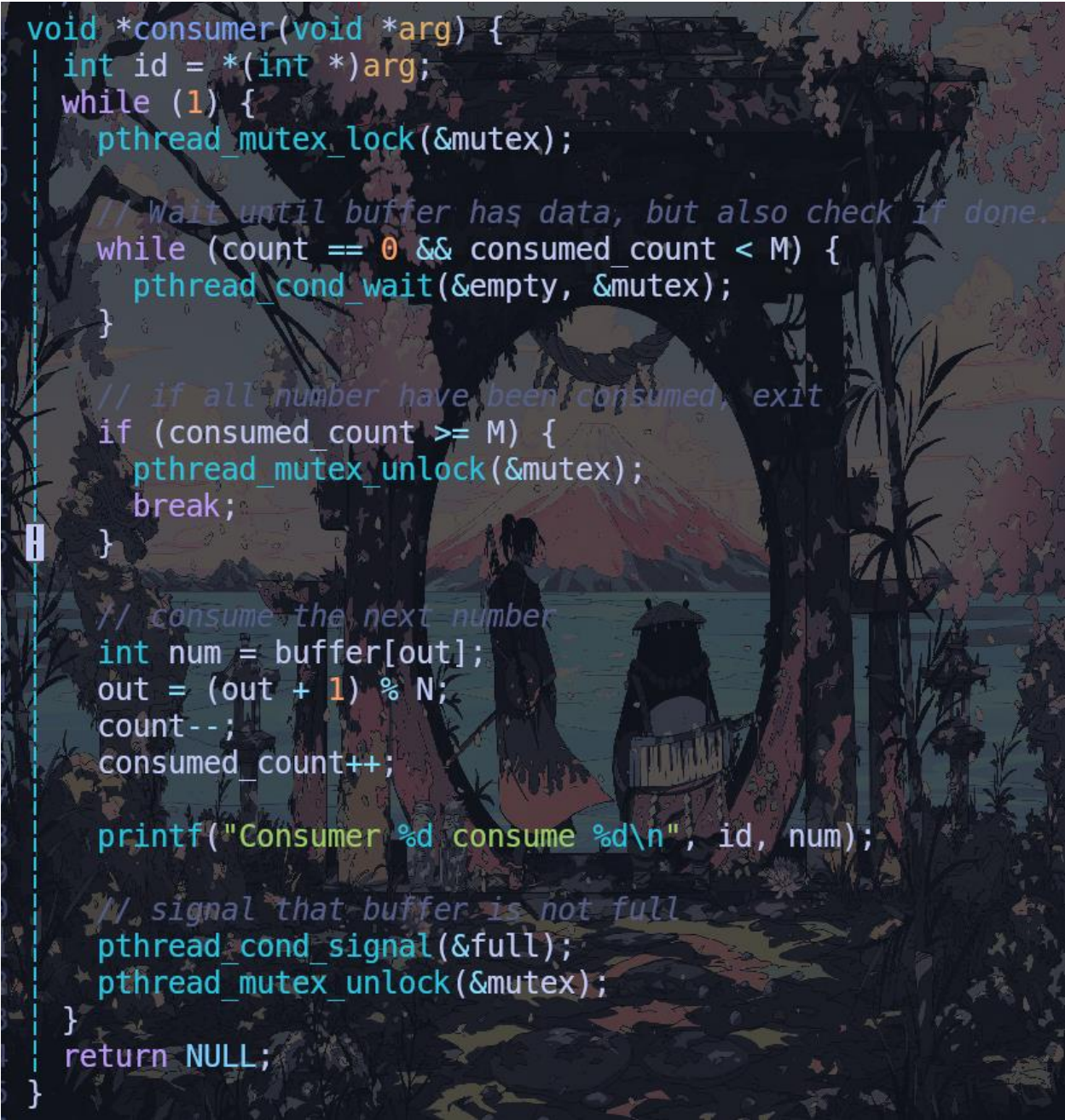
```
// wait for all producers to finish
for (int i = 0; i < P; i++) {
    pthread_join(producers[i], NULL);
}

// Wait for all consumers to finish
for (int i = 0; i < C; i++) {
    pthread_join(consumers[i], NULL);
}
```



```
4 void *producer(void *arg) {  
8     int id = *(int *)arg;  
2     free(arg);  
1     while (1) {  
9         pthread_mutex_lock(&mutex);  
9         while (count == N) {  
8             pthread_cond_wait(&full, &mutex);  
7         }  
6         // if all numbers produced, exit  
5         if (produced_count >= M) {  
8             pthread_mutex_unlock(&mutex);  
3             break;  
2         }  
  
1         // produce next number  
1         int num = produced_count;  
2         produced_count++;  
  
4         // insert into buffer  
5         buffer[in] = num;  
6         in = (in + 1) % N;  
7         count++;  
  
9         printf("Producer %d produced %d\n", id, num);  
  
1        // Signal that buffer is not empty  
2        pthread_cond_signal(&empty);  
3        pthread_mutex_unlock(&mutex);  
4    }  
5    return NULL;  
6 }
```

Producer implementation.



```
void *consumer(void *arg) {
    int id = *(int *)arg;
    while (1) {
        pthread_mutex_lock(&mutex);

        // wait until buffer has data, but also check if done.
        while (count == 0 && consumed_count < M) {
            pthread_cond_wait(&empty, &mutex);
        }

        // if all number have been consumed, exit
        if (consumed_count >= M) {
            pthread_mutex_unlock(&mutex);
            break;
        }

        // consume the next number
        int num = buffer[out];
        out = (out + 1) % N;
        count--;
        consumed_count++;

        printf("Consumer %d consume %d\n", id, num);

        // signal that buffer is not full
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Consumer implementation.

Usage: ./master-worker, M (Total number) N (Maximum shared buffer) C (Number of consumers) P (Number of producers)

A terminal window showing the execution of the ./master-worker 10 3 2 1 command. The output displays a sequence of 10 produced items by Producer 0 and 3 consumed items by Consumer 0 and 1. The background of the terminal is a dark, artistic illustration of a Japanese landscape with a mountain, water, and figures.

```
iron@Ubuntu:~/cs4375-0S/Concurrency$ ./master-worker 10 3 2 1
Producer 0 produced 0
Producer 0 produced 1
Producer 0 produced 2
Consumer 0 consumed 0
Consumer 0 consumed 1
Consumer 0 consumed 2
Producer 0 produced 3
Producer 0 produced 4
Consumer 1 consumed 3
Consumer 1 consumed 4
Producer 0 produced 5
Consumer 0 consumed 5
Producer 0 produced 6
Consumer 0 consumed 6
Producer 0 produced 7
Consumer 0 consumed 7
Producer 0 produced 8
Consumer 0 consumed 8
Producer 0 produced 9
Consumer 0 consumed 9
All producers finished. Exiting.
```

Figure Small Test: ./master-worker 10 3 2 1

A terminal window showing the execution of the ./master-worker 1 1 1 1 command. The output shows 1 produced item by Producer 0 and 1 consumed item by Consumer 0. The background is a dark, textured image.

```
iron@Ubuntu:~/cs4375-0S/Concurrency$ ./master-worker 1 1 1 1
Producer 0 produced 0
Consumer 0 consumed 0
All producers finished. Exiting.
```

Figure Edge case ./master-worker 1 1 1 1



```
Producer 0 produced 43
Producer 0 produced 44
Producer 0 produced 45
Producer 0 produced 46
Producer 0 produced 47
Consumer 2 consumed 40
Producer 2 produced 48
Producer 2 produced 49
Consumer 4 consumed 41
Consumer 4 consumed 42
Consumer 4 consumed 43
Consumer 4 consumed 44
Consumer 4 consumed 45
Consumer 4 consumed 46
Consumer 4 consumed 47
Consumer 4 consumed 48
Consumer 4 consumed 49
All producers finished. Exiting.
iron@Ubuntu:~/cs4375-0S/Concurrency$
```

Figure Larger test ./master-worker 50 10 5 4

The image shows a terminal window with a dark background and a Japanese-style illustration. The illustration features a person in traditional clothing standing under a large, dark, leafy archway, looking out over a body of water with a small boat. The text in the terminal is white and black, showing the output of a program with multiple producers and consumers. The output consists of 20 lines of 'produced' and 'consumed' messages from three different producers and consumers, followed by a final 'Exiting.' message. The terminal prompt is 'iron@Ubuntu:~/cs4375-0S/Concurrency\$ |'.

```
Consumer 1 consumed 3
Consumer 1 consumed 4
Producer 0 produced 5
Producer 1 produced 6
Producer 1 produced 7
Producer 1 produced 8
Producer 1 produced 9
Consumer 1 consumed 5
Consumer 1 consumed 6
Consumer 1 consumed 7
Consumer 1 consumed 8
Consumer 1 consumed 9
Producer 0 produced 10
Producer 0 produced 11
Producer 0 produced 12
Producer 1 produced 13
Producer 1 produced 14
Consumer 0 consumed 10
Consumer 0 consumed 11
Consumer 0 consumed 12
Consumer 0 consumed 13
Consumer 0 consumed 14
Producer 1 produced 15
Producer 1 produced 16
Producer 1 produced 17
Producer 1 produced 18
Producer 1 produced 19
Consumer 0 consumed 15
Consumer 2 consumed 16
Consumer 2 consumed 17
Consumer 2 consumed 18
Consumer 2 consumed 19
All producers finished. Exiting.
iron@Ubuntu:~/cs4375-0S/Concurrency$ |
```

Figure Multiple producers: ./master-worker 20 5 3 2

The image shows a terminal window with a dark background featuring a faint, artistic illustration of a Japanese landscape. The illustration includes a traditional building with a thatched roof, a person in a kimono standing near a body of water, and a mountain in the distance. The terminal text is white and consists of a series of log messages from a program named 'Big Big Test'. The messages show consumers (IDs 35, 43, 10, 38) consuming items (IDs 99969 to 99999) and finally reporting that all producers have finished and the program is exiting. The prompt at the bottom indicates the user is 'iron' on an 'Ubuntu' machine in the directory '~/cs4375-0S/Concurrency\$'.

```
Consumer 35 consumed 99969
Consumer 35 consumed 99970
Consumer 35 consumed 99971
Consumer 35 consumed 99972
Consumer 35 consumed 99973
Consumer 35 consumed 99974
Consumer 35 consumed 99975
Consumer 35 consumed 99976
Consumer 35 consumed 99977
Consumer 35 consumed 99978
Consumer 43 consumed 99979
Consumer 35 consumed 99980
Consumer 35 consumed 99981
Consumer 35 consumed 99982
Consumer 35 consumed 99983
Consumer 35 consumed 99984
Consumer 35 consumed 99985
Consumer 35 consumed 99986
Consumer 35 consumed 99987
Consumer 35 consumed 99988
Consumer 10 consumed 99989
Consumer 35 consumed 99990
Consumer 35 consumed 99991
Consumer 35 consumed 99992
Consumer 35 consumed 99993
Consumer 35 consumed 99994
Consumer 35 consumed 99995
Consumer 35 consumed 99996
Consumer 35 consumed 99997
Consumer 35 consumed 99998
Consumer 38 consumed 99999
All producers finished. Exiting.
iron@Ubuntu:~/cs4375-0S/Concurrency$
```

Figure Big Big Test: ./master-worker 100000 100 50 20

USAGE:

```
iron@Ubuntu:~/cs4375-OS/Concurrency$ ./master-worker
Usage: ./master-worker, M (Total number) N (Maximum shared buffer size) C (Number of consumers) P (number of producers)
iron@Ubuntu:~/cs4375-OS/Concurrency$
```

Challenging Aspects

Preventing deadlocks, the mutex is always release before waiting before waiting on condition variables, and the signaling is done while holding the mutex to avoid race conditions. I use while loops around `pthread_cond_wait` to re-check the condition.

Producers stop when `produced_count >= M`. Consumers stop when `consumed_count >= M`. After all producers exit, consumers may still be waiting on empty; they will be woken by the last producer's signal and then exit because `consumed_count >= M`.

Buffer overflow/underflow, the condition `count == N` and `count == 0` prevent any illegal access.

Order of production, numbers are produced in increasing order, but consumption can be interleaved due to multiple consumers.

References:

<https://www.tutorialspoint.com/article/producer-oconsumer-problem-in-c>

<https://hpc-tutorials.llnl.gov/posix/>

<https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>

https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

https://www.daemon-systems.org/man/pthread_cond_init.3.html

https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html

https://man7.org/linux/man-pages/man3/pthread_create.3.html

https://man7.org/linux/man-pages/man3/pthread_join.3.html

<https://comp.anu.edu.au/courses/comp2310/labs/09-conc1/>

<https://courses.cs.washington.edu/courses/cse333/20su/lectures/lec-23/lec-23-code/cthreads.c>

Operating Systems: Three Easy Pieces Chapter 30